

CSC 429 Assignment 2

Adrian Aldridge

For the coding problems I wrote the python in VS code. The VS code extensions I use to run Python are: Python, Pylance, Python debugger, and Python Environments.

To run the code, open the code folder in a VS code environment, click run while in main.py and each problem should print with paths and traversal order. That is the only environment I've confirmed that it runs in.

Problem 1 – Graph and Grid BFS and DFS

For this problem I implemented Generic BFS and DFS functions that can accept both the graph and grid representations. I defined the graph as a list of pairs of connected edges. The grid is defined as a 2D list. I defined start and goal positions for the graph and the grid. In main, I ran the BFS and DFS functions on both the graph and the grid.

For the graph, BFS and DFS returned the same path: A -> B -> C -> E

This is likely due to the graph being very simple. The DFS function did have a slightly longer traversal order, due to it exploring the D-F path first before hitting the C-E path. In this case, either algorithm would work, and both return the shortest path possible. However, with larger graphs the difference would be more visible. BFS favors finding the shortest path possible but can be memory intensive. DFS is better for if you need to find any path to the goal or have memory constraints.

For the grid, BFS and DFS also found the same path: (6, 0) -> (6, 1) -> (6, 2) -> (5, 2) -> (4, 2) -> (4, 3) -> (3, 3)

(6, 0) in this grid is the start position, and (3, 3) is the goal position. I used 2D indexing to travel through the list.

In this case, BFS and DFS found the same path to the goal, and it is the shortest path. BFS had a longer traversal order due to exploring level by level, but it was guaranteed to find the shortest path. DFS in this case, got lucky. DFS just happened to explore the shortest path first going from (6, 0) to (6, 1), if instead the DFS algorithm had gone to (5, 0) it would have gone down a much

longer path to the goal position. DFS did have a much shorter traversal order but could have very easily gone into that pitfall of finding a much longer and more inefficient path.

Problem 2 – Grid DLS

For this problem, I implemented a generic DLS function. It uses many of the same features as the DFS function but with an implemented depth limit. It also prints out a complete trace of the steps taken and paths abandoned. It wasn't necessarily required but I liked to see the complete trace with abandoned paths. It helped me keep track of backtracking and really see the process DLS took. I ran DLS on the same grid from Problem 1 using two depth limits: 6 and 10

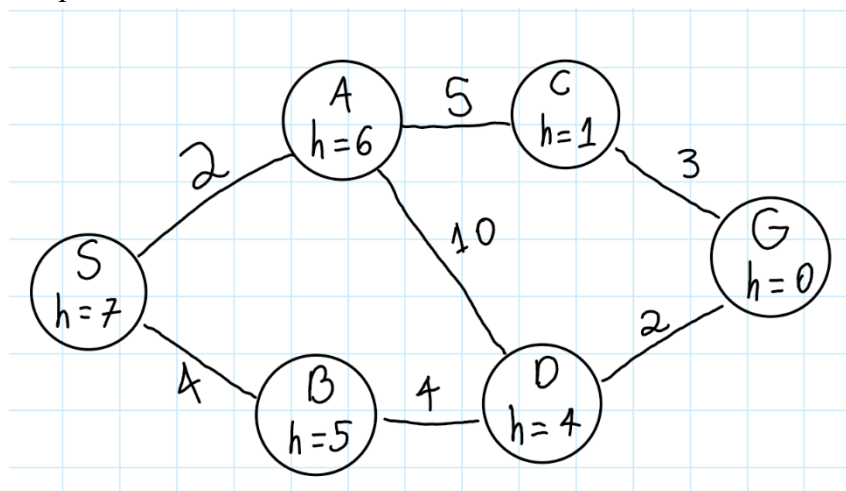
DLS with limit 6: (6, 0) -> (6, 1) -> (6, 2) -> (5, 2) -> (4, 2) -> (4, 3) -> (3, 3)

DLS with limit 10: (6, 0) -> (5, 0) -> (4, 0) -> (3, 0) -> (2, 0) -> (1, 0) -> (1, 1) -> (1, 2) -> (1, 3) -> (2, 3) -> (3, 3)

Here we can see that depth limit 6 did find the most efficient path, very similar to how BFS and DFS performed. In the full trace I did see it went down the (5, 0) path first and ended up abandoning it, which lead to a very heavy memory load and long traversal order. In contrast, depth limit 10 just took the longer path as the goal was within the depth limit. It only had to abandon one path and backtrack once. This shows the importance of picking a depth limit, running with a lower depth limit could lead to finding no path or finding a very efficient path like with depth limit 6. Running with a larger depth limit runs much like an unlimited DFS, it can turn up longer paths but has less backtracking and a lower memory load.

Problem 3 – A* on a Weighted Graph

Graph:



A* Search:

Start with a frontier containing the start state and an empty explored,

- Frontier: S(7)
- Explored:

Expand S and move it to explored, add neighbors to frontier

A* uses $f(n) = g(n) + h(n)$ so cost will be heuristic + actual

- Frontier: A($6 + 2 = 8$), B($5 + 4 = 9$)
- Explored: S(7)

A has the lower cost so we explore A, and add its neighbors to the frontier

- Frontier: B(9), C($2 + 5 + 1 = 8$), D($2 + 10 + 4 = 16$)
- Explored: S(7), A(8)

The lowest cost here is C, so we explore C

- Frontier: B(9), D(16), G($2 + 5 + 3 + 0 = 10$)
- Explored: S(7), A(8), C(8)

B is the next lowest value, so we explore B. Through B we have a better path to D, so we update D's cost.

- Frontier: G(10), D($4 + 4 + 4 = 12$)
- Explored: S(7), A(8), C(8), B(9)

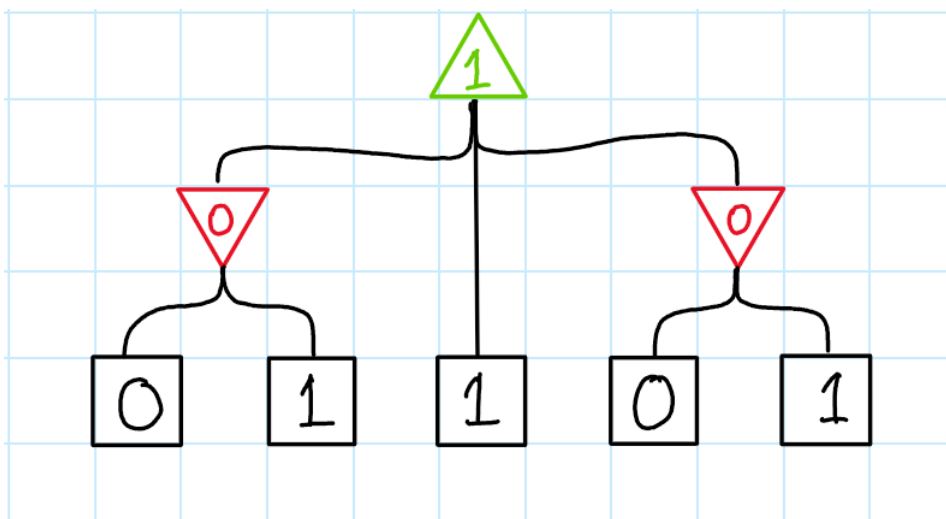
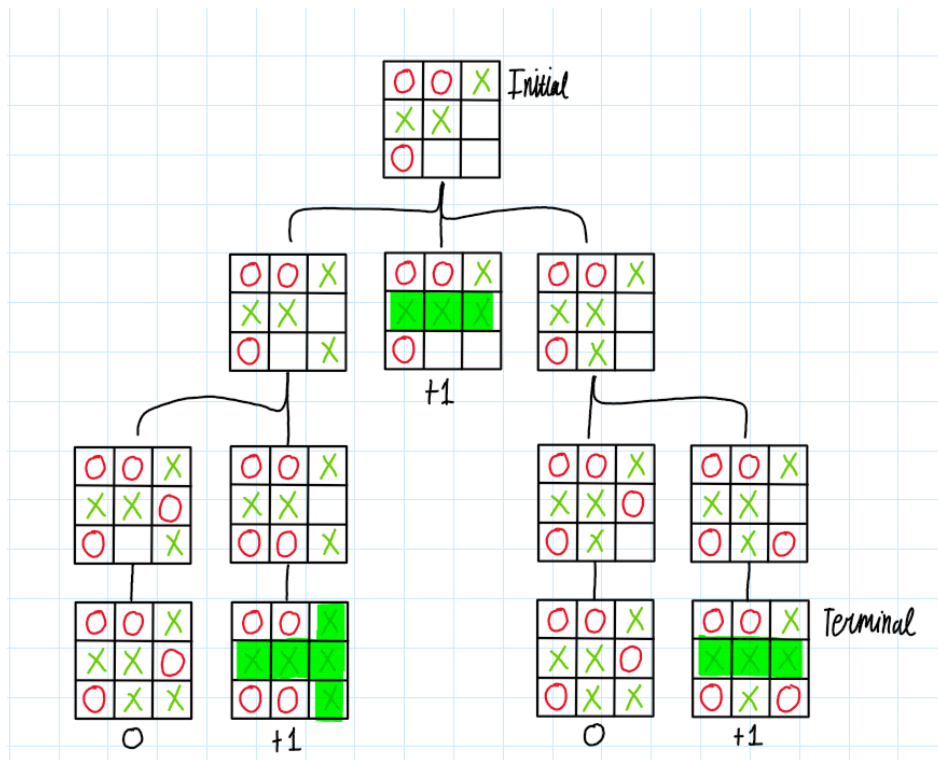
G has the next lowest value, so we take the path to G through C.

- Frontier: D(12)
- Explored: S(7), A(8), C(8), B(9), G(10)

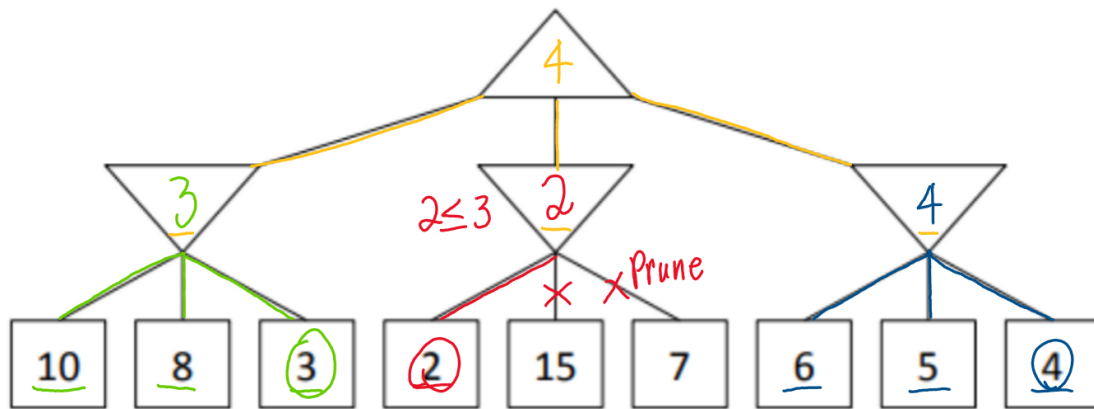
We have reached the goal, the final path is S -> A -> C -> G with a final cost of 10

Problem 4 – Tic Tac Toe Minimax

The game tree has 4 layers with 5 terminal states



Problem 5 – Minimax Tree



In this minimax tree, the branches containing 15 and 7 can be pruned. Because, when reading the tree from left to right, our first min node is 3. Then when reading the second min node we get a value of 2, which is less than three. Any other value adopted by that min node is guaranteed to be less than 3. Because the max node wants to pick the highest value, and 3 is our current highest value we do not need to read anything guaranteed to be less than 3.